Ring Buffer Guide: Scrolling a Tall Level Through 512 Lines of Chip RAM

A companion guide to Building an AMIGA Game Engine

Richard Shannon

AmigaGameEngine v1.0

Contents

1. Terms 2

2. Memory Layout 2

3. Where a World Line Lives 2

4. Why the Mirror Half Exists 2

5. The Drawing Rules 3
- Rule 1: A moving object goes on the copy that is on screen . . . 3
- Rule 2: Erase, tint and overlay at the line the object was drawn at . . . 3
- Rule 3: Background changes go into both halves . . . 3
- Rule 4: Never address a single copy with `Y & 255` . . . 3
- Rule 5: Only write resident rows . . . 3

6. Which Rows Are Resident 3

7. Streaming a Row 4

8. Full Rebuild 4

9. The Camera 4

10. Limits 5

11. Checklist for Code That Touches the Buffer 5

A Duck Rescue level can be up to 50 tile rows tall (800 lines); Level 6 is 43 rows (688 lines). The display window shows 216 lines. Holding a whole level as a bitmap would cost 50 rows × 2,560 bytes = 128,000 bytes per buffer, and the engine needs two buffers (`DisplayScreen` and `NonDisplayScreen`): 256,000 bytes, half of an A500's 512 KB of Chip RAM.

Instead, each buffer holds only the **16 tile rows around the camera** in a ring, plus an exact copy of those 16 rows directly below them. That is 512 lines, or 81,920 bytes per buffer (163,840 bytes for both). The level map itself stays in Fast RAM as tile indices, and rows are drawn into the ring as the camera reaches them.

This guide explains the layout, how a world position becomes a buffer address, how rows stream in, and the addressing rules that every routine drawing into the buffer must follow. **Read it before writing anything that draws into `DisplayScreen` or `NonDisplayScreen`.** The most confusing bugs in this engine came from getting one of those rules wrong.

Source: `engine/graphics/tmx_renderer.asm` (camera, streaming, rebuild) and `engine/include/engine_const.i` (the `TILEMAP_*` and `RING_BUFFER_*` constants). The Blit Order Guide covers *when* things are drawn; this guide covers *where*.

1. Terms

Term	Meaning
World Y	A line of the whole level: 0 is the top of the map. Row *R* covers world lines 16*R* to 16*R* + 15.
Camera Y	`TilemapCameraY`: the world line at the top of the display window. Clamped to `LevelMinCameraY..LevelMaxCameraY`.
Screen Y	World Y − camera Y: 0 is the top line of the display window, 215 the bottom.
Buffer line	One of the 512 lines of a screen buffer.
Buffer row	16 buffer lines. Rows 0–15 are the **primary half**; rows 16–31 are the **mirror half**, an exact copy of rows 0–15.
Resident rows	The 16 world rows currently held in the ring: `RingBufferTopRow..RingBufferBotRow`.
`DisplayScreen`	The buffer the display shows. Every visible pixel is here.
`NonDisplayScreen`	A clean copy of the background (tiles and water, no moving objects). Erasing an object means copying its rectangle back from here.

2. Memory Layout

The game screen has 4 bitplanes (16 colours), stored **interleaved**: each line holds plane 0's 40 bytes, then plane 1's, plane 2's and plane 3's.

Constant	Value	Meaning
`SCREEN_WIDTH_BYTE`	40	bytes per plane per line (320 pixels)
`TILEMAP_LINE_STRIDE`	160	bytes per line, all four planes
`TILEMAP_SCREEN_MOD`	120	`BPL1MOD`/`BPL2MOD`: skip the other three planes
`TILEMAP_ROW_STRIDE`	2,560	bytes per tile row (16 lines)
`RING_BUFFER_ROWS`	16	primary rows
`RING_BUFFER_MASK`	255	world line to primary buffer line
`RING_BUFFER_ROW_MASK`	15	world row to buffer row
`RING_BUFFER_ALLOC_SIZE`	81,920	bytes per buffer (512 lines × 160)

Plane *p* of buffer line *L* starts at `buffer + L*160 + p*40`. One Blitter pass covers all four planes of a rectangle by treating each line as four lines of 40 bytes: a 16-line tile is a 64-line blit.

3. Where a World Line Lives

World row *R* is stored in buffer row `R & 15`, and again 16 rows lower. Because 256 is a power of two, the arithmetic is a mask:

```
primary buffer line = Y & 255          (0..255)
mirror  buffer line = (Y & 255) + 256  (256..511)
```

Both copies hold the same pixels for the background. Which one the viewer sees depends on the camera.

4. Why the Mirror Half Exists

The display reads 216 consecutive buffer lines, starting at `cam & 255`. The bitplane pointers in `cpPlanes` are set to `DisplayScreen + (cam & 255) × 160` (plus 40 per plane) by `TilemapApplyCameraY`.

The start line is at most 255, so the last displayed line is at most 255 + 215 = 470: the window never runs off the end of the 512-line buffer. **The display never wraps,** so no Copper split is needed. Any part of the window below buffer line 255 is read from the mirror half, which holds the same rows.

A 256-line ring without the mirror would need a mid-screen Copper split to jump back to line 0 wherever the window crossed line 255. Objects straddling that seam would also have to be drawn in two pieces. The mirror costs 40,960 bytes per buffer and removes both problems.

Worked example: Level 6 at the bottom. Level 6 is 43 rows tall, so the camera's maximum is 43 × 16 − 216 = 472.

```
cam = 472           cam & 255 = 216
display reads buffer lines 216 .. 431

world line 472 (screen Y 0)    -> buffer line 216  (primary half)
world line 511 (screen Y 39)   -> buffer line 255  (primary half, last line)
world line 512 (screen Y 40)   -> buffer line 256  (mirror half)
world line 687 (screen Y 215)  -> buffer line 431  (mirror half)
```

Most of that screen is shown from the mirror half. A routine that draws world line 687 at `687 & 255` = 175 writes to a line the viewer is **not** looking at. That is exactly how the player once stayed dry underwater at the bottom of Level 6: the tint went to the primary copy while the display showed the mirror.

5. The Drawing Rules

Rule 1: A moving object goes on the copy that is on screen

Objects that are erased and redrawn every frame (player, enemies, friends, moving crates) are drawn once, at the buffer line the display is showing:

```
draw line = (cam & 255) + (Y - cam)        ; may be 256 or more: the mirror half
```

`RingBufferCamY` holds `cam & 255`. Use this line, not `Y & 255`.

Rule 2: Erase, tint and overlay at the line the object was drawn at

The camera can move between one frame's draw and the next frame's erase. Anything that later touches an object (the erase, a water tint, a stun overlay) must use the line the draw used, not a line recomputed from the new camera. The player records the camera its draw used in `PrevPlayerCameraY`. `TilemapErasePlayer` and `TilemapSubmergePlayer` both compute `(PrevPlayerCameraY & 255) + (PrevY − PrevPlayerCameraY)`.

Rule 3: Background changes go into both halves

Anything that changes the background permanently writes the primary **and** the mirror copy, because either might be on screen next frame. This covers tiles, water lines and restored tiles. Examples:

- `TilemapStreamTileRow`: new rows, both buffers, both halves;
- `TilemapApplyWaveScanline`: water lines, both buffers, both halves;
- `RestoreBackgroundTile` and `TilemapErase16x16Actor`: tile-aligned restores, both halves;
- `TilemapBlitSingleTile`: both halves of `DisplayScreen`.

Rule 4: Never address a single copy with `Y & 255`

`Y & 255` is only correct as the first half of a "write both halves" pair. For a single write it picks the primary copy whether or not that copy is on screen.

Rule 5: Only write resident rows

A world row outside `RingBufferTopRow..RingBufferBotRow` is not in the ring: its buffer row currently holds a different world row. `TilemapApplyWaveScanline` bounds-checks against the resident rows for this reason. Writing a non-resident row draws onto whatever row shares its slot.

6. Which Rows Are Resident

`TilemapCheckRowStream` keeps the resident rows around the visible window with one row of lookahead:

```
wanted top    = (cam - 16) >> 4          (clamped at 0)
wanted bottom = (cam + 216 + 15) >> 4
```

The visible window needs at most 15 rows: 216 lines starting part-way into a row. The 16th resident row is lookahead on whichever side the camera last moved toward. Scrolling down streams the next row below and gives up the top lookahead row, which shares its buffer slot. Scrolling up does the reverse.

Variable	Meaning
`RingBufferTopRow` / `RingBufferBotRow`	First and last resident world rows (always 16 rows, inclusive)
`RingBufferCamY`	`cam & 255`: buffer line at the top of the display
`CameraCullTopY` / `CameraCullBottomY`	`cam - 32` and `cam + 248`: objects outside are not drawn

7. Streaming a Row

When the camera crosses into a new row, `TilemapCheckRowStream` moves the resident range one row and calls `TilemapStreamTileRow` for the row that just became resident. That row is always outside the visible window, so streaming never changes a pixel the viewer can see.

`TilemapStreamTileRow` for map row *R* (buffer row *b* = `R & 15`):

1. Clear buffer row *b* of `NonDisplayScreen` (2,560 bytes, CPU `TurboClear`).
2. Draw every tile layer of row *R* into it (`TilemapRenderOneRowAllLayers`): background, platform, ladder, foreground and water layers, or the level's own layer list.
3. If the row is below `WaterPixelY`, apply the wave and fill to its submerged lines (`TilemapApplyWaveScanline`).
4. Copy row *b* to `DisplayScreen` (one 64 × 20-word blit).
5. Copy row *b* to row *b* + 16 in `NonDisplayScreen`.
6. Copy row *b* to row *b* + 16 in `DisplayScreen`.

A fast fall can cross several rows in one frame, so the stream loops until the resident range catches up. A jump of 16 rows or more cannot be streamed and triggers a full rebuild instead.

8. Full Rebuild

`TilemapRebuildRingBuffer` redraws the ring from scratch around the camera:

1. Clear all 81,920 bytes of `NonDisplayScreen`.
2. Set the resident rows to `cam >> 4` .. `cam >> 4` + 15 and draw all 16 (`TilemapDrawInitialRingBuffer`).
3. Copy the primary half to the mirror half (two 512-line blits).
4. Clone all of `NonDisplayScreen` to `DisplayScreen` (four 512-line blits).
5. Re-apply the water (`TilemapRenderWaterInBuffer`).

A rebuild **repaints the visible rows**, and it wipes every moving and static object off `DisplayScreen`. It is only used where the whole view changes anyway:

Caller	Why
`TilemapInit` (level start)	First fill; the display is still hidden by the level transition
`TilemapCentreCamera` (respawn)	The camera jumps to the respawn point
`TilemapCheckRowStream`	The camera moved 16 rows or more in one step
`UndoMove` (via the camera snap)	A rewind restored a camera far from the current one

After a rebuild, the caller must put back what the rebuild does not draw. Mark crates dirty and pickups undrawn so they redraw, and clear the player's "previously drawn" state so its next erase does not copy garbage. `PlayerRespawn` in `player_water.asm` is the worked example.

9. The Camera

`TilemapUpdateCamera` runs once a frame in the logic, after the draw pass (see the Blit Order Guide). It follows the player with a dead zone:

Setting	Default	Per-level TMX property
Top margin (`LevelCamMarginTop`)	16 (`CAM_MARGIN_TOP`)	`cam_margin_top`
Bottom margin (`LevelCamMarginBottom`)	176 (`CAM_MARGIN_BOTTOM`)	`cam_margin_bottom`
Minimum camera Y	0	`min_camera_y`
Maximum camera Y	rows × 16 − 216	`max_camera_y` (never less than the default)
Start camera	centred on the player	`initial_camera_y` / `initial_row`

The camera moves when the player leaves the band between the margins. It moves 2 lines a frame when walking or climbing, at the player's fall speed when falling, and settles the last odd line when the player stops. Level start and respawn instead centre the player (`TilemapCentredCameraY`: player Y − `CAM_CENTRE_OFFSET`, 100, clamped).

`TilemapApplyCameraY` stores the new camera and recomputes the culling bounds and `RingBufferCamY`. It rewrites the four bitplane pointers in `cpPlanes`, rebuilds the Copper sky gradient (`TilemapUpdateCopperSky`) and refreshes the player's frame (`GameShowPlayerHook`).

The Copper reloads `cpPlanes` at line 0, so **a camera change takes effect next frame**. Everything drawn this frame was placed for the camera the display is still showing.

10. Limits

Limit	Value	Where
Map height	13 to 50 rows	`MIN_MAP_ROWS`/`MAX_MAP_ROWS` in `export_level.py`; `MAX_SCREEN_ROWS` in `engine_const.i`
Map width	20 columns	`MAX_MAP_WIDTH`; the screen is exactly 20 tiles wide, with no horizontal scrolling
Per-level tables	sized for 20 × 64	`MAX_GAME_MAP_SIZE` (1,280): `GameMap`, `DirtyTiles`, the undo map copy; `LiveWaterMap` holds 16-bit tile words, so 8 + 2 × 1,280 bytes
Rows streamed per frame	any number below 16	a jump of 16 or more rebuilds

A table sized by the default level height (`TILEMAP_MAP_TILES`, 42 rows) instead of `MAX_GAME_MAP_SIZE` overflows on a taller level. That is what corrupted the Player record on Level 6. Size every per-level table from `MAX_GAME_MAP_SIZE`, and clamp row loops with `CurrentMapHeight`, never a constant.

11. Checklist for Code That Touches the Buffer

1. **Is it background or a moving object?** Background goes into both halves (rule 3); a moving object goes into the copy on screen (rule 1).
2. **Does it touch something drawn earlier?** Use the line that draw used, saved with the camera at the time (rule 2).
3. **Is the row resident?** Check against `RingBufferTopRow`..`RingBufferBotRow` (rule 5).
4. **Does it repaint visible rows?** Then it is a rebuild: redraw every object afterwards (§8).
5. **Test at the bottom of Level 6,** where the display shows the mirror half, and while the camera scrolls across buffer line 255.